Onkar Jadhav

AI / Full-Stack Software Engineer
Pune, Maharashtra, India
Phone: +91-7058604679 | Email: onkarj012@gmail.com
GitHub: github.com/Onkarj012 | LinkedIn: linkedin.com/in/onkarj012 | Portfolio: leo4.dev

Summary

Final-year Computer Science student (9.08 CGPA) with end-to-end ownership across production AI systems, systems programming, and full-stack product engineering. Shipped Vaani, a notarised macOS dictation product with native C++/Obj-C addons, from concept to distributable release independently. Built Lore Engine, a production RAG platform with hybrid BM25 + pgvector retrieval over 6,000+ wiki pages and deploy-gated evaluation harnesses. Designed StockXpert, a hierarchical deep learning prediction system achieving a 12% improvement in short-horizon return correlation across all 100 Nifty NSE equities. Open-sourced GovBench, an LLM governance benchmark evaluated across US, India, and EU legal systems. Currently building Stride, an AI-first adaptive wellness platform with five domain agents and multi-modal input, and PromptForge, an agentic prompt-engineering pipeline with cross-model benchmarking. Proficient in Python, TypeScript, React, Next.js, FastAPI, Node.js, TensorFlow, RAG architectures, and agentic AI systems.

Education

Bachelor of Technology in Computer Science Nov 2022 – Jun 2026
Pimpri Chinchwad College of Engineering, Savitribai Phule Pune University *CGPA: 9.08 (up to 7th semester)*

- Relevant Coursework: Machine Learning, Data Structures & Algorithms, Database Management Systems, Operating Systems, Computer Networks, Object-Oriented Programming.

Higher Secondary Certificate Jun 2022
Shri Mhalsakant Junior College, Maharashtra State Board *Score: 89.17%*

Experience

Stride — AI-First Adaptive Wellness Platform April 2026 – Present
Founder & Lead Engineer *Remote*

- Built and shipped Stride to production as sole engineer; running live with an early cohort of **5–10 active trial users** driving iterative product feedback.
- Architected a six-layer AI-first system: multi-modal input capture (text, voice, image, barcode, nutrition labels), a behavioral memory layer tracking preferences and adherence, an NLU layer for structured extraction, five specialised domain agents (diet, workout, sleep, hydration, habits) sharing memory, an analytics layer, and an adaptive UX layer driven by behavioral signals.
- Separated concerns by design: LLMs handle understanding and intent extraction while a deterministic four-component calculator (BMR, NEAT, EAT, TEF) handles computation — eliminating opaque LLM arithmetic errors while retaining natural-language input.
- Built natural-language workout parsing converting free text such as "30 min run, 3x10 squats at 60kg" into structured sets/reps/duration/MET values without rigid input templates.
- Selected Convex for real-time reactive state over REST + WebSockets, and OpenRouter for multi-provider LLM routing to avoid vendor lock-in across five concurrent agent types.

Engineering Projects

Lore Engine — RAG Platform for Fan Wiki Knowledge Bases 2026 – Present
Next.js, FastAPI, PostgreSQL, pgvector, BM25, Celery, Redis, Cloudflare R2, S3, OpenAI, Anthropic Claude

- Architected a full-stack RAG application ingesting **6,000+ MediaWiki pages** per series into a hybrid retrieval pipeline combining BM25 lexical search and pgvector dense embeddings via reciprocal rank fusion (RRF) — specifically chosen over pure vector search to handle proper-noun disambiguation and exact character-name matching that semantic models fail on.

- Designed section-aware chunking at H2/H3 wiki boundaries (not naive fixed token windows) with enriched [Series][Page][Section] metadata per chunk; per-chunk first_appearance_order spoiler tags assigned at ingestion time and filtered at retrieval time, ensuring users never surface lore beyond their current story progress.
- Chose pgvector on a single Postgres instance over dedicated vector stores (Pinecone, Weaviate, Qdrant) to scale to 5M vectors per series while eliminating infrastructure complexity and keeping retrieval within one network hop.
- Designed an async ingestion pipeline (Celery workers, Redis broker, Cloudflare R2/S3 object storage) that fetches pages from the MediaWiki API, converts wikitext to structured markdown, stores raw pages in R2/S3, and persists infobox metadata in PostgreSQL.
- Established a **50-question per-series ground-truth eval harness** measuring retrieval MRR and end-to-end answer accuracy, gating every deploy; validated architecture against the Harry Potter wiki (6,000 pages).
- Shipped two distinct consumer surfaces on a single shared retriever: a streaming chat interface with inline citations (conversational) and a hierarchically generated lore book with aggressive per-series caching (reference).

StockXpert — AI-Powered Deep Learning Equity Prediction Platform Nov 2025 – Present
Python, TensorFlow, FastAPI, Next.js, TypeScript, Tailwind CSS, Cloudflare R2, AWS

- Architected a hierarchical deep learning system combining ResNet, GRU, Bi-LSTM, and multi-head attention to model cross-horizon price dependencies across all **100 Nifty NSE equities**, outperforming single-architecture baselines on directional accuracy.
- Constructed a leakage-free feature pipeline from raw OHLCV data incorporating RSI, MACD, ADX, Bollinger Bands, SMA/EMA distance, and volatility indicators with strict temporal train/validation/test splits to prevent lookahead bias in feature engineering.
- Designed a multi-objective Huber loss with directional regularisation and tail-event oversampling, achieving a **12% improvement in short-horizon return correlation** over baseline and reducing directional error on high-volatility equities.
- Built a snapshot-first serving layer on Cloudflare R2 with market-aware TTL caching, delivering **sub-second API latency across all 100 Nifty stocks** during market hours while eliminating redundant inference cost.
- Delivered a full-stack trading dashboard (Next.js + FastAPI) with live stock screener, real-time alerts, OHLCV candlestick charts, technical indicator overlays, and 1D–10D forecast tables with calibrated confidence scores.

GovBench — Open-Source LLM Governance & Bias Evaluation Framework Mar 2026 – Present
Python, LLM Evaluation Pipelines, Sentiment Analysis, Hallucination Detection, Adversarial Testing, AI Safety *github.com/Onkarj012*

- Designed and open-sourced an LLM evaluation framework assessing model readiness for judicial and governmental deployment across **6 pillars**: demographic bias, procedural integrity, corruption resistance, jurisdictional awareness, transparency, and minority protection — pillars chosen because MMLU-style benchmarks expose none of these failure modes.
- Implemented controlled demographic isolation testing across **12 identity variants** per scenario with naturalistic (non-leading) prompting, measuring inherent bias in bail, sentencing, welfare, and immigration decisions across **US, India, and EU** legal systems.
- Evaluated 4 production LLMs (Claude Sonnet 4.6, Gemini Flash Lite, DeepSeek V4, GPT-OSS 120B) across baseline, pressure, and adversarial stress modes; top performers scored **90%+ overall** while revealing concrete failures in minority protection (GPT-OSS: **63%**) and jurisdictional awareness (DeepSeek V4: **56.7%**).
- Built an automated scoring pipeline combining sentiment variance analysis, position drift tracking, and hallucination detection to emit composite deployment-readiness grades, enabling unattended evaluation on new model releases.

PromptForge — Agentic Prompt Engineering & Benchmarking Platform Jan 2026 – Feb 2026
Node.js, Express.js, BullMQ, Convex, OpenRouter, React.js

- Architected a creator–critic–evaluator agentic pipeline for iterative prompt improvement, scoring clarity, completeness, constraint coverage, and token efficiency across versioned revision trails — treating prompts as testable, regression-tracked artifacts.
- Implemented cross-model routing and benchmarking across OpenAI, Anthropic, Gemini, and open-source models via OpenRouter, enabling objective quality, latency, and cost-per-output comparisons for data-driven model selection.
- Built a fault-tolerant async backend using BullMQ workers with idempotent retries and Convex persistence, supporting reliable prompt regression testing at scale without job loss on LLM provider timeouts.
- Emitted structured prompt outputs with metadata: token budget, latency profile, expected output shape, and version changelog — enabling prompts to be reviewed and deployed like code.

Technical Skills

Languages: Python, TypeScript, JavaScript, SQL, C++, Objective-C, Java
Frontend: React.js, Next.js, Tailwind CSS, HTML, CSS, Electron
Backend: FastAPI, Node.js, Express.js, BullMQ, Celery, Redis, REST APIs, WebSockets
AI / ML: TensorFlow, RAG Pipelines, pgvector, BM25, Reciprocal Rank Fusion, LLM Evaluation, Agentic Workflows, Behavioral Memory, Multi-modal AI, Hallucination Detection, XGBoost, OpenRouter
LLM Providers: Anthropic Claude, OpenAI GPT, Gemini, Groq, Deepgram Nova-2, whisper.cpp (local)
Databases: PostgreSQL (+ pgvector), MongoDB, Convex
Cloud & DevOps: AWS, Cloudflare R2, Git, GitHub, macOS Distribution (notarisation, code signing)
Native / Systems: C++, Objective-C, Node-API (native addons), Electron, macOS Accessibility API
System Design: Event-driven architecture, async job pipelines, snapshot-first serving, hybrid retrieval, multi-agent coordination, adaptive UX systems